

Лабораторная работа №4

Работа с репозиториями git: Gitflow и общепринятые коммиты

Липатников Андрей

2026-10-07

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Работа с репозиториями git».

В ходе работы установлен пакет git-flow и инструменты для семантического версионирования на базе Node.js, выполнен практический сценарий работы с тестовым репозиторием: создание репозитория, инициализация git-flow, формирование релизов 1.0.0 и 1.2.3 с журналом изменений и публикацией релизов на GitHub.

Сделаны выводы о достижении поставленной цели.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
1.4	Объект и предмет исследования	7
1.5	Методы исследования	7
1.6	Схема выполнения работы	7
2	Теоретическая часть	9
2.1	Рабочий процесс Gitflow	9
2.2	Семантическое версионирование	10
2.3	Общепринятые коммиты (Conventional Commits)	10
3	Материалы и методы	13
4	Выполнение лабораторной работы	14
4.1	Установка программного обеспечения	14
4.2	Установка инструментов общепринятых коммитов	16
4.3	Создание репозитория	16
4.4	Конфигурация общепринятых коммитов	17
4.5	Конфигурация git-flow	19
4.6	Создание релиза 1.0.0	20
4.7	Разработка новой функциональности	23
4.8	Создание релиза 1.2.3	23
5	Результаты	26
6	Обсуждение результатов	27
7	Заключение	28
	Список литературы	29
A	Что выносится в приложения	30

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	8
4.1	Включение репозитория Copr	14
4.2	Установка git-flow	14
4.3	Установка Node.js	15
4.4	Установка npnm	15
4.5	npnm setup и source ~/.bashrc	15
4.6	Установка commitizen	16
4.7	Установка standard-changelog	16
4.8	Создание репозитория на GitHub	17
4.9	Первый коммит и отправка на GitHub	17
4.10	Инициализация package.json	18
4.11	Файл package.json после конфигурации	18
4.12	git add, git cz, git push	19
4.13	Инициализация git-flow	19
4.14	Проверка ветки	20
4.15	Отправка всех веток на GitHub	20
4.16	Установка вышестоящей ветки	20
4.17	Создание релиза 1.0.0	21
4.18	Журнал изменений для первого релиза	21
4.19	Завершение релиза 1.0.0	22
4.20	Отправка веток после релиза 1.0.0	22
4.21	Отправка тегов после релиза 1.0.0	22
4.22	Создание релиза v1.0.0 на GitHub	22
4.23	Создание функциональной ветки	23
4.24	Завершение работы с функциональной веткой	23
4.25	Создание релиза 1.2.3 и обновление версии в package.json	24
4.26	Генерация журнала изменений	24
4.27	Обновление журнала изменений	24
4.28	Завершение релиза 1.2.3	25
4.29	Отправка веток после релиза 1.2.3	25
4.30	Отправка тегов после релиза 1.2.3	25
4.31	Создание релиза v1.2.3 на GitHub	25

Список таблиц

2.1	Основные типы коммитов Conventional Commits	11
-----	---	----

1 Введение

1.1 Актуальность

Современная разработка программного обеспечения ведётся командами, где одновременно идут доработка текущего выпуска и подготовка следующего. Без строгой модели ветвления история проекта превращается в запутанный набор коммитов, а выпуски теряют воспроизводимость. Модель Gitflow упорядочивает этот процесс отдельными ветками для функций, релизов и исправлений, а общепринятые коммиты (Conventional Commits) в связке с семантическим версионированием позволяют автоматически вести журнал изменений и присваивать версии. Для курса «Операционные системы» эти навыки обязательны: весь практикум сдаётся через git-репозиторий, и правильная структура веток с подписанными релизами — часть отчётности.

1.2 Цель работы

Получение навыков правильной работы с репозиториями git.

1.3 Задачи

1. Установить программное обеспечение: git-flow, Node.js, pnpm, commitizen, standard-changelog.

2. Выполнить работу для тестового репозитория: создать репозиторий git-extended и подключить его к GitHub.
3. Преобразовать рабочий репозиторий в репозиторий с git-flow и общепринятыми коммитами: инициализировать git-flow, настроить формат коммитов через commitizen, сформировать релизы 1.0.0 и 1.2.3 с журналом изменений.

1.4 Объект и предмет исследования

- **Объект исследования:** процессы управления версиями и организации рабочего процесса в системе контроля версий git.
- **Предмет исследования:** применение модели ветвления Gitflow, общепринятых коммитов и семантического версионирования в тестовом репозитории git-extended.

1.5 Методы исследования

В работе использованы следующие методы:

- установка и настройка программного обеспечения через пакетный менеджер dnf и пакетный менеджер rpm;
- практический эксперимент: последовательность команд git и git-flow над тестовым репозиторием;
- анализ результатов: проверка ветвей, тегов и релизов на GitHub.

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий при выполнении работы.

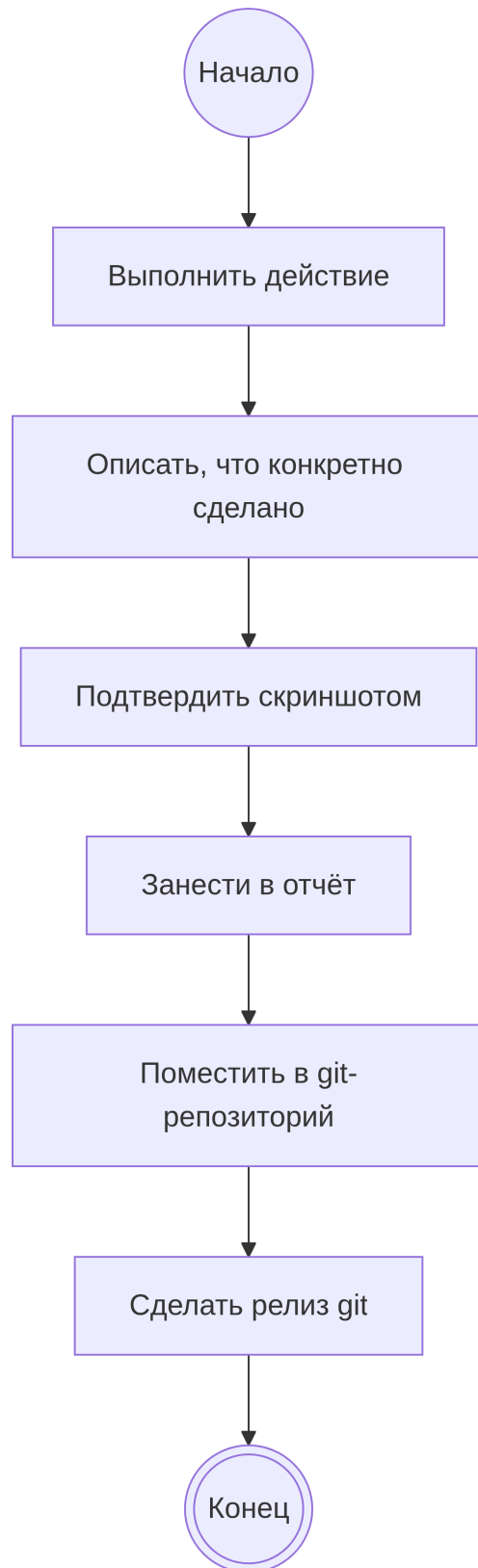


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

2.1 Рабочий процесс Gitflow

Рабочий процесс Gitflow, опубликованный и популяризованный Винсентом Дриссенем, предполагает выстраивание строгой модели ветвления с учётом выпуска проекта. Модель ориентирована на рабочий процесс на основе релизов и включает отдельную ветку для исправлений ошибок в рабочей среде.

В модели Gitflow используются две основные ветви: в ветке `master` хранится официальная история релиза (всем коммитам рекомендуется присваивать номер версии), а ветка `develop` предназначена для объединения всех функций. Из ветки `develop` создаются функциональные ветки `feature` и ветки выпуска `release`; по завершении работы ветка `feature` сливается с `develop`, а ветка `release` — с ветками `develop` и `master`. Если в `master` обнаружена проблема, из неё создаётся ветка исправления `hotfix` — единственная ветка, создаваемая непосредственно от `master`; по завершении она сливается с `develop` и `master`, и `master` помечается обновлённым номером версии. Наличие отдельных веток позволяет одной команде дорабатывать текущий выпуск, пока другая готовит функции для следующего, а исправления вносить, не прерывая рабочий процесс.

Инициализация структуры в существующем репозитории выполняется командой `git flow init`; для GitHub параметр `Version tag prefix` устанавливается в `v`.

2.2 Семантическое версионирование

Версия программного обеспечения задаётся в виде кортежа МАЖОРНАЯ_ВЕРСИЯ.МИНОРНАЯ_ВЕРСИЯ.ПАТЧ. Номер версии увеличивается: мажорную версию — при обратно несовместимых изменениях API, минорную — при добавлении новой функциональности без нарушения обратной совместимости, патч-версию — при обратно совместимых исправлениях. Для предрелизных и билд-метаданных допускаются дополнительные обозначения. На практике применяются комплексные продукты, использующие информацию из коммитов системы версионирования, — поэтому коммиты должны иметь стандартизованный вид.

2.3 Общепринятые коммиты (Conventional Commits)

Спецификация Conventional Commits — соглашение о том, как писать сообщения коммитов, совместимое с семантическим версионированием. Структура коммита:

```
<тип>(<область>): <описание изменения>
<пустая линия>
[необязательное тело]
<пустая линия>
[необязательный нижний колонтитул]
```

Заголовок обязателен; любая строка сообщения не может быть длиннее 100 символов; тема описывает изменение кратко, в повелительном наклонении настоящего времени, без заглавной первой буквы и точки в конце. Тело включает мотивацию изменения и противопоставление предыдущему поведению; нижний колонтитул содержит информацию о критических изменениях, внешние

ссылки и ссылку на закрываемую issue. Критические изменения помечаются префиксом `BREAKING CHANGE:`.

Основные типы коммитов приведены в табл. 2.1.

Таблица 2.1: Основные типы коммитов Conventional Commits

Тип	Назначение	Соответствие SemVer
<code>fix:</code>	исправление ошибки в коде	<code>PATCH</code>
<code>feat:</code>	новая функциональность	<code>MINOR</code>
<code>BREAKING CHANGE:</code>	изменение, нарушающее обратную совместимость	<code>MAJOR</code>
<code>revert:</code>	отмена предыдущей фиксации	—
<code>docs:</code> , <code>style:</code> , <code>refactor:</code> , <code>perf:</code> , <code>test:</code> , <code>build:</code> , <code>ci:</code> , <code>chore:</code>	документация, форматирование, рефакторинг, производительность, тесты, сборка, CI	—

Соглашение Angular требует дополнительные типы: `build` (система сборки, внешние зависимости), `ci` (конфигурация CI), `docs` (документация), `perf` (производительность), `refactor`, `style`, `test`. Областью действия должно быть имя затронутого пакета, с исключениями `packaging` и `changelog`.

Более подробно про Git и Bash см. в [1–3].

3 Материалы и методы

- **Среда выполнения:** Fedora 41 (Sway) на виртуальной машине VirtualBox (настроена в лабораторной работе №1).
- **Устанавливаемое ПО:** git-flow (из репозитория Copr), Node.js и pnpm, глобальные пакеты commitizen и standard-changelog, утилита gh.
- **Тестовый репозиторий:** git-extended на GitHub, подключённый по SSH.
- **Методы:** последовательное выполнение команд из методических указаний с фиксацией каждого шага скриншотом; проверка состояния ветвей, тегов и релизов.

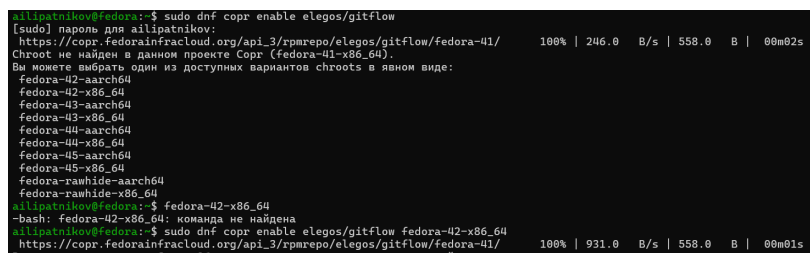
4 Выполнение лабораторной работы

4.1 Установка программного обеспечения

4.1.1 Установка git-flow

Включён репозиторий Copr и установлен пакет git-flow (рис. 4.1, рис. 4.2):

```
dnf copr enable elegos/gitflow
dnf install gitflow
```



```
aikipatnikov@fedora:~$ sudo dnf copr enable elegos/gitflow
[sudo] пароль для aikipatnikov:
https://copr.fedorainfracloud.org/api_3/rpmrepo/elegos/gitflow/fedora-41/ 100% | 246.0 B/s | 558.0 B | 00m02s
Chroot не найден в данном проекте Copr (fedora-41-x86_64).
Вы можете выбрать один из доступных вариантов chroots в явном виде:
fedora-42-x86_64
fedora-43-aarch64
fedora-43-x86_64
fedora-44-aarch64
fedora-44-x86_64
fedora-45-aarch64
fedora-45-x86_64
fedora-rawhide-aarch64
fedora-rawhide-x86_64
aikipatnikov@fedora:~$ sudo dnf copr enable elegos/gitflow fedora-42-x86_64
https://copr.fedorainfracloud.org/api_3/rpmrepo/elegos/gitflow/fedora-41/ 100% | 931.0 B/s | 558.0 B | 00m01s
```

Рисунок 4.1: Включение репозитория Copr



```
aikipatnikov@fedora:~$ sudo dnf install gitflow
Обновление и загрузка репозитория:
Fedora 41 - x86_64 - Updates 100% | 1.8 KiB/s | 3.4 KiB | 00m02s
Fedora 41 - x86_64 100% | 11.8 KiB/s | 3.4 KiB | 00m00s
Copr repo for gitflow owned by elegos 100% | 2.4 KiB/s | 2.4 KiB | 00m01s
```

Рисунок 4.2: Установка git-flow

4.1.2 Установка Node.js и npm

Установлены Node.js и npm (рис. 4.3, рис. 4.4):

```
dnf install nodejs
dnf install pnpm
```

```
ailipatnikov@fedora:~$ sudo dnf install nodejs
```

Обновление и загрузка репозитория:				
Репозитории загружены.				
Пакет	Арх.	Версия	Репозиторий	Размер
Установка: nodejs	x86_64	1:22.21.1-2.fc41	updates	161.7 KiB
Установка зависимостей:				
nodejs-libs	x86_64	1:22.21.1-2.fc41	updates	77.0 MiB
Установка слабых зависимостей:				
nodejs-devel	noarch	1:22.21.1-2.fc41	updates	97.5 MiB
nodejs-full-i18n	x86_64	1:22.21.1-2.fc41	updates	30.4 MiB
nodejs-npm	x86_64	1:10.9.4-1.22.21.1.2.fc41	updates	9.2 MiB

Рисунок 4.3: Установка Node.js

```
ailipatnikov@fedora:~$ sudo dnf install pnpm
```

Обновление и загрузка репозитория:				
Репозитории загружены.				
Пакет	Арх.	Версия	Репозиторий	Размер
Установка: pnpm	noarch	10.9.0-1.fc41	updates	16.3 MiB

Рисунок 4.4: Установка pnpm

4.1.3 Настройка Node.js

Выполнена настройка окружения: каталог с исполняемыми файлами, устанавливаемыми yarn, добавлен в переменную PATH (рис. 4.5):

```
pnpm setup
source ~/.bashrc
```

```
ailipatnikov@fedora:~$ pnpm setup
Appended new lines to /home/ailipatnikov/.bashrc

Next configuration changes were made:
export PNPM_HOME="/home/ailipatnikov/.local/share/pnpm"
case ":$PATH:" in
  *:$PNPM_HOME:*) ;;
  *) export PATH="$PNPM_HOME:$PATH" ;;
esac

To start using pnpm, run:
source /home/ailipatnikov/.bashrc
ailipatnikov@fedora:~$ source ~/.bashrc
```

Рисунок 4.5: pnpm setup и source ~/.bashrc

4.2 Установка инструментов общепринятых коммитов

Установлены глобально commitizen (для помощи в форматировании коммитов; устанавливается скрипт git-cz) и standard-changelog (для создания журналов изменений) (рис. 4.6, рис. 4.7):

```
pnpm add -g commitizen
pnpm add -g standard-changelog
```



Рисунок 4.6: Установка commitizen

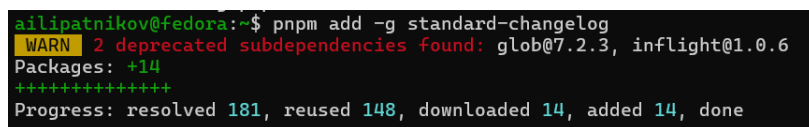


Рисунок 4.7: Установка standard-changelog

4.3 Создание репозитория

Создан репозиторий git-extended на GitHub (рис. 4.8). Выполнен первый коммит, репозиторий подключён как удалённый и отправлен на GitHub (рис. 4.9):

```
git commit -m "first commit"
git remote add origin git@github.com:<username>/git-extended.git
git push -u origin master
```

```
ailipatnikov@fedora:~$ mkdir -p ~/git-extended
ailipatnikov@fedora:~$ cd ~/git-extended
ailipatnikov@fedora:~/git-extended$ git init
Инициализирован пустой репозиторий Git в /home/ailipatnikov/git-extended/.git/
ailipatnikov@fedora:~/git-extended$ echo "# git-extended" > README.md
ailipatnikov@fedora:~/git-extended$ git add .
ailipatnikov@fedora:~/git-extended$
```

Рисунок 4.8: Создание репозитория на GitHub

```
ailipatnikov@fedora:~/git-extended$ git commit -m "first commit"
[master (корневой коммит) 94021a3] first commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
ailipatnikov@fedora:~/git-extended$ git remote add origin git@github.com:pakarah/git-extended.git
ailipatnikov@fedora:~/git-extended$ git push -u origin master
Enter passphrase for key '/home/ailipatnikov/.ssh/id_ed25519':
```

Рисунок 4.9: Первый коммит и отправка на GitHub

4.4 Конфигурация общепринятых коммитов

Инициализирована конфигурация для пакетов Node.js (рис. 4.10):

```
pnpm init
```

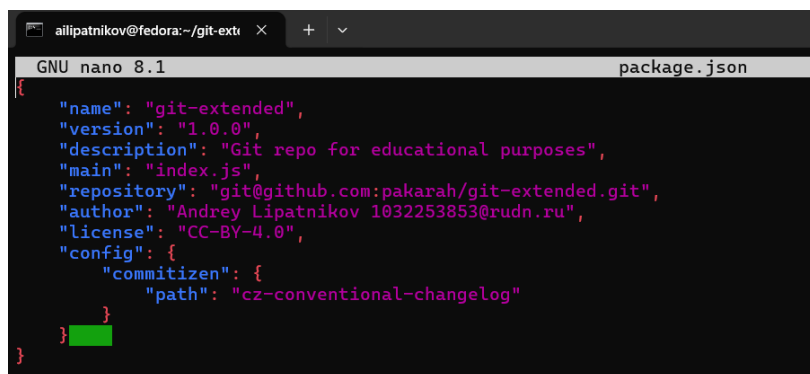
Заполнены параметры пакета: название, лицензия CC-BY-4.0. В файл `package.json` добавлена команда для формирования коммитов (рис. 4.11):

```
"config": {
  "commitizen": {
    "path": "cz-conventional-changelog"
  }
}
```

```
ailipatnikov@fedora:~/git-extended$ pnpm init
Wrote to /home/ailipatnikov/git-extended/package.json

{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "packageManager": "pnpm@10.9.0"
}
```

Рисунок 4.10: Инициализация package.json



```
ailipatnikov@fedora:~/git-exti x + v
GNU nano 8.1 package.json
{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "Git repo for educational purposes",
  "main": "index.js",
  "repository": "git@github.com:pakarah/git-extended.git",
  "author": "Andrey Lipatnikov 1032253853@rudn.ru",
  "license": "CC-BY-4.0",
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}
```

Рисунок 4.11: Файл package.json после конфигурации

Новые файлы добавлены в индекс, выполнен коммит через git-cz, изменения отправлены на GitHub (рис. 4.12):

```
git add .
git cz
git push
```

```

ailipatnikov@fedora:~/git-extended$ git add .
ailipatnikov@fedora:~/git-extended$ git cz
cz-cli@4.3.2, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: feat:      A new feature
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 94 chars):
(20) 4th lab com "git cz"
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
[master 9b4555a] feat: 4th lab com "git cz"
1 file changed, 14 insertions(+)
 create mode 100644 package.json
ailipatnikov@fedora:~/git-extended$ git push
Enter passphrase for key '/home/ailipatnikov/.ssh/id_ed25519':
Перечисление объектов: 4, готово.
Подсчет объектов: 100% (4/4), готово.
При сжатии изменений используется до 2 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (3/3), 1.20 КиБ | 136.00 КиБ/с, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:pakarah/git-extended.git
 94021a3..9b4555a master -> master
ailipatnikov@fedora:~/git-extended$

```

Рисунок 4.12: git add, git cz, git push

4.5 Конфигурация git-flow

Инициализирован git-flow, префикс для ярлыков установлен в v (рис. 4.13). Проверена текущая ветка (рис. 4.14): после инициализации работа идёт в ветке develop.

```

git flow init
git branch

```

```

ailipatnikov@fedora:~/git-extended$ git flow init -f

Which branch should be used for bringing forth production releases?
- develop
- master
Branch name for production releases: [master]

Which branch should be used for integration of the "next release"?
- develop
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/home/ailipatnikov/git-extended/.git/hooks]
ailipatnikov@fedora:~/git-extended$

```

Рисунок 4.13: Инициализация git-flow

```
ailipatnikov@fedora:~/git-extended$ git branch
* develop
master
```

Рисунок 4.14: Проверка ветки

Весь репозиторий загружен в хранилище, внешняя ветка установлена как вышестоящая для ветки develop (рис. 4.15, рис. 4.16):

```
git push --all
git branch --set-upstream-to=origin/develop develop
```

```
ailipatnikov@fedora:~/git-extended$ git push --all
Enter passphrase for key '/home/ailipatnikov/.ssh/id_ed25519':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/pakarah/git-extended/pull/new/develop
remote:
To github.com:pakarah/git-extended.git
 * [new branch]      develop -> develop
```

Рисунок 4.15: Отправка всех веток на GitHub

```
ailipatnikov@fedora:~/git-extended$ git branch --set-upstream-to=origin/develop develop
branch 'develop' set up to track 'origin/develop'.
```

Рисунок 4.16: Установка вышестоящей ветки

4.6 Создание релиза 1.0.0

Создан релиз с версией 1.0.0 (рис. 4.17):

```
git flow release start 1.0.0
```

Создан журнал изменений первой генерацией standard-changelog, добавлен в индекс и зафиксирован (рис. 4.18):

```
standard-changelog --first-release
git add CHANGELOG.md
git commit -am 'chore(site): add changelog'
```

```
ailipatnikov@fedora:~/git-extended$ git flow release start 1.0.0
Переключились на новую ветку «release/1.0.0»

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.0.0'
```

Рисунок 4.17: Создание релиза 1.0.0

```
ailipatnikov@fedora:~/git-extended$ standard-changelog --first-release
ailipatnikov@fedora:~/git-extended$ git add CHANGELOG.md
ailipatnikov@fedora:~/git-extended$ git commit -am 'chore(site): add changelog'
[release/1.0.0 57f2df9] chore(site): add changelog
1 file changed, 5 insertions(+)
create mode 100644 CHANGELOG.md
```

Рисунок 4.18: Журнал изменений для первого релиза

Релизная ветка слита с основной (рис. 4.19), данные отправлены на GitHub вместе с тегами (рис. 4.20, рис. 4.21):

```
git flow release finish 1.0.0
git push --all
git push --tags
```

```

ailipatnikov@fedora:~/git-extended$ git flow release finish 1.0.0
Branches 'master' and 'origin/master' have diverged.
And local branch 'master' is ahead of 'origin/master'.
Уже на «master»
Ваша ветка опережает «origin/master» на 2 коммита.
(Используйте «git push», чтобы опубликовать ваши локальные коммиты)
Переключились на ветку «develop»
Эта ветка соответствует «origin/develop».
Merge made by the 'ort' strategy.
  CHANGELOG.md | 5 +++++
  1 file changed, 5 insertions(+)
  create mode 100644 CHANGELOG.md
Ветка release/1.0.0 удалена (была 57f2df9).

Summary of actions:
- Release branch 'release/1.0.0' has been merged into 'master'
- The release was tagged 'v1.0.0'
- Release tag 'v1.0.0' has been back-merged into 'develop'
- Release branch 'release/1.0.0' has been locally deleted
- You are now on branch 'develop'
ailipatnikov@fedora:~/git-extended$ |

```

Рисунок 4.19: Завершение релиза 1.0.0

```

ailipatnikov@fedora:~/git-extended$ git push --all
Enter passphrase for key '/home/ailipatnikov/.ssh/id_ed25519':
Перечисление объектов: 6, готово.
Подсчет объектов: 100% (6/6), готово.
При сжатии изменений используется до 2 потоков
Сжатие объектов: 100% (5/5), готово.
Запись объектов: 100% (5/5), 2.92 КиБ | 374.00 КиБ/с, готово.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:pakarah/git-extended.git
  9b4555a..dc91b9e  develop -> develop
  9b4555a..48855fb  master -> master

```

Рисунок 4.20: Отправка веток после релиза 1.0.0

```

ailipatnikov@fedora:~/git-extended$ git push --tags
Enter passphrase for key '/home/ailipatnikov/.ssh/id_ed25519':
Перечисление объектов: 1, готово.
Подсчет объектов: 100% (1/1), готово.
Запись объектов: 100% (1/1), 187 байтов | 187.00 КиБ/с, готово.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:pakarah/git-extended.git
 * [new tag]          v1.0.0 -> v1.0.0

```

Рисунок 4.21: Отправка тегов после релиза 1.0.0

Создан релиз на GitHub с журналом изменений в описании (рис. 4.22):

```
gh release create v1.0.0 -F CHANGELOG.md
```

```

ailipatnikov@fedora:~/git-extended$ gh release create v1.0.0 -F CHANGELOG.md
https://github.com/pakarah/git-extended/releases/tag/v1.0.0

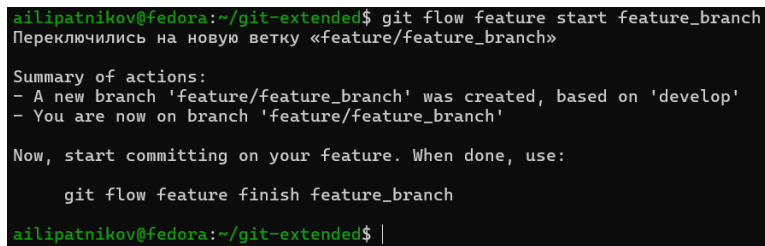
```

Рисунок 4.22: Создание релиза v1.0.0 на GitHub

4.7 Разработка новой функциональности

Создана ветка для новой функциональности; после завершения работы ветка объединена с develop (рис. 4.23, рис. 4.24):

```
git flow feature start feature_branch
git flow feature finish feature_branch
```



```
ailipatnikov@fedora:~/git-extended$ git flow feature start feature_branch
Переключились на новую ветку «feature/feature_branch»

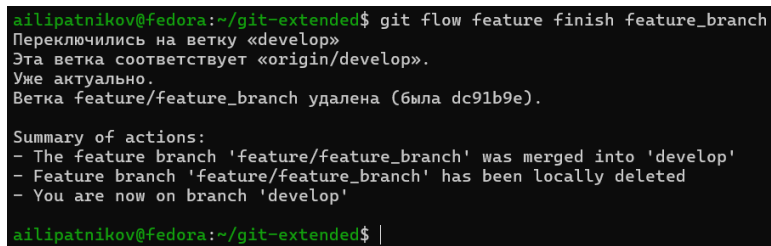
Summary of actions:
- A new branch 'feature/feature_branch' was created, based on 'develop'
- You are now on branch 'feature/feature_branch'

Now, start committing on your feature. When done, use:

    git flow feature finish feature_branch

ailipatnikov@fedora:~/git-extended$ |
```

Рисунок 4.23: Создание функциональной ветки



```
ailipatnikov@fedora:~/git-extended$ git flow feature finish feature_branch
Переключились на ветку «develop»
Эта ветка соответствует «origin/develop».
Уже актуально.
Ветка feature/feature_branch удалена (была dc91b9e).

Summary of actions:
- The feature branch 'feature/feature_branch' was merged into 'develop'
- Feature branch 'feature/feature_branch' has been locally deleted
- You are now on branch 'develop'

ailipatnikov@fedora:~/git-extended$ |
```

Рисунок 4.24: Завершение работы с функциональной веткой

4.8 Создание релиза 1.2.3

Создан релиз с версией 1.2.3, номер версии обновлён в файле package.json (рис. 4.25):

```
git flow release start 1.2.3
```

Обновлена версия в package.json — установлена в 1.2.3 (рис. 4.25).


```
ailipatnikov@fedora:~/git-extended$ git flow release finish 1.2.3
Переключились на ветку «master»
Эта ветка соответствует «origin/master».
Merge made by the 'ort' strategy.
```

Рисунок 4.28: Завершение релиза 1.2.3

```
ailipatnikov@fedora:~/git-extended$ git push --all
Enter passphrase for key '/home/ailipatnikov/.ssh/id_ed25519':
Перечисление объектов: 9, готово.
Подсчет объектов: 100% (9/9), готово.
При сжатии изменений используется до 2 потоков
Сжатие объектов: 100% (6/6), готово.
Запись объектов: 100% (6/6), 2.92 КиБ | 598.00 КиБ/с, готово.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:pakarah/git-extended.git
 944f542..67a318e develop -> develop
 0300444..713dcff master -> master
```

Рисунок 4.29: Отправка веток после релиза 1.2.3

```
ailipatnikov@fedora:~/git-extended$ git push --tags
Enter passphrase for key '/home/ailipatnikov/.ssh/id_ed25519':
Перечисление объектов: 1, готово.
Подсчет объектов: 100% (1/1), готово.
Запись объектов: 100% (1/1), 188 байтов | 188.00 КиБ/с, готово.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:pakarah/git-extended.git
 * [new tag]          v1.2.3 -> v1.2.3
ailipatnikov@fedora:~/git-extended$ |
```

Рисунок 4.30: Отправка тегов после релиза 1.2.3

Создан релиз v1.2.3 на GitHub с комментарием из журнала изменений (рис. 4.31):

```
gh release create v1.2.3 -F CHANGELOG.md
```

```
ailipatnikov@fedora:~/git-extended$ gh release create v1.2.3 -F CHANGELOG.md
https://github.com/pakarah/git-extended/releases/tag/v1.2.3
ailipatnikov@fedora:~/git-extended$ |
```

Рисунок 4.31: Создание релиза v1.2.3 на GitHub

5 Результаты

В ходе работы получены следующие результаты:

- установлено программное обеспечение: git-flow (из Copr), Node.js, pnpm, commitizen, standard-changelog;
- создан тестовый репозиторий git-extended и подключён к GitHub;
- репозиторий преобразован в репозиторий с git-flow (префикс тегов v) и общепринятыми коммитами (cz-conventional-changelog в package.json);
- созданы и опубликованы релизы: теги v1.0.0 и v1.2.3 на GitHub с журналом изменений CHANGELOG.md в описании;
- выполнен полный цикл работы с функциональной веткой: feature start и feature finish.

6 Обсуждение результатов

Последовательность выполнения соответствует методическим указаниям: модель ветвления Gitflow развёрнута на тестовом репозитории, коммиты формируются в стандартизованном виде через `commitizen`, релизы нумеруются по семантическому версионированию и автоматически сопровождаются журналом изменений `standard-changelog`. Разделение ветвей показало главное свойство модели: исправления и релизы не пересекаются с текущей разработкой — после `git flow release start 1.0.0` в ветке релиза допустима только отладка и документация, новые функции туда не попадают. Связка типов коммитов с компонентами версии (`fix` → `PATCH`, `feat` → `MINOR`, `BREAKING CHANGE` → `MAJOR`) позволяет в дальнейшем вести версионирование автоматически. Замеченных отклонений от ожидаемого поведения команд не зафиксировано.

7 Заключение

- Цель работы достигнута: получены навыки правильной работы с репозиториями git.
- Освоены модель ветвления Gitflow (ветки master, develop, feature, release, hotfix) и инструмент git-flow.
- Освоены общепринятые коммиты (Conventional Commits) и семантическое версионирование; сформированы и опубликованы релизы v1.0.0 и v1.2.3 с журналом изменений.
- Полученные навыки применимы ко всем последующим работам курса: репозиторий оформлен по стандарту, релизы публикуются утилитой gh.

Список литературы

1. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
3. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.

А Что выносятся в приложения

При необходимости сюда выносятся:

- листинги программ;
- большие таблицы с исходными данными;
- промежуточные выкладки;
- акты внедрения и т.п.

Каждое приложение начинается с новой страницы и имеет заголовок (например, «Приложение А. Листинг программы»).